StyleSense: Feature Engineering

This notebook builds new analysis-ready columns from the five cleaned StyleSense datasets.

Why feature engineering comes after cleaning: Although cleaned data is *correct*, engineered features make it *useful for deeper analysis*. The new columns added here require business logic, cross-dataset joins, or statistical transformations.

What i covered:

1. Time-based features (Sales)
2. Transaction-level profit metrics (Sales)
3. Product feature join (Sales ← Products)
4. Customer RFM scoring (Customers)
5. Stock health & days until stockout (Inventory)
6. Fixed vs variable expense classification (Expenses)

In [1]:
```
# Library imports
import pandas as pd          # data manipulation and analysis
import numpy as np            # numerical operations
import matplotlib.pyplot as plt  # plotting
import seaborn as sns         # statistical visualisations built on matplotlib

sns.set_theme(style="whitegrid")   # clean white background for all charts
plt.rcParams['figure.dpi'] = 110   # sharper figure output

# Consistent colour palette used across all charts in this notebook
BLUE   = "#1f77b4"
GREEN  = "#2ca02c"
ORANGE = "#ff7f0e"
RED    = "#d62728"
PURPLE = "#9467bd"
```

In [2]:
```
# Load all five cleaned datasets
# parse_dates tells pandas to convert date columns to datetime objects automatically,
# which lets us do date arithmetic (e.g. extract month, calculate days between dates)
sales     = pd.read_csv('sales_clean_data.csv',     parse_dates=['date'])
```

```
customers = pd.read_csv('customers_clean_data.csv', parse_dates=['first_purchase_date', 'last_purchase_date'])
products  = pd.read_csv('products_clean_data.csv')
expenses  = pd.read_csv('expenses_clean_data.csv',  parse_dates=['date'])
inventory = pd.read_csv('inventory_clean_data.csv', parse_dates=['date'])
```

In [15]:
```
# Quick sanity check: confirm each dataset loaded with the expected number of rows and columns
print(f"sales:     {sales.shape}")
print(f"customers: {customers.shape}")
print(f"products:  {products.shape}")
print(f"expenses:  {expenses.shape}")
print(f"inventory: {inventory.shape}")
```

```
sales: (9989, 19)
customers: (200, 17)
products: (25, 12)
expenses: (1322, 4)
inventory: (13675, 12)
```

Check columns for each dataset

In [16]:
```
# List all column names in the sales dataset
print(f"sales:\n{sales.columns.tolist()}")
```

```
sales:Index(['sale_id', 'date', 'store_channel', 'payment_method', 'product_id',
       'product_name', 'category', 'sub_category', 'quantity_sold',
       'unit_price', 'discount_pct', 'discount_amount', 'total_sale',
       'cost_total', 'gross_profit', 'customer_id', 'is_promotion',
       'is_returned', 'net_profit'],
      dtype='str')
```

In [17]:
```
# List all column names in the customers dataset
print(f"customers:\n{customers.columns.tolist()}")
```

```
customers:Index(['customer_id', 'name', 'email', 'age', 'gender', 'city', 'state',
       'segment', 'total_items_purchased', 'total_spent', 'num_orders',
       'last_purchase_date', 'favourite_category', 'first_purchase_date',
       'avg_order_value', 'age_group', 'derived_segment'],
      dtype='str')
```

In [18]:
```
# List all column names in the products dataset
print(f"products:\n{products.columns.tolist()}")
```

```
products:Index(['product_id', 'product_name', 'category', 'sub_category', 'base_price',
       'cost_price', 'season', 'profit_per_unit', 'profit_margin_pct',
       'is_winter', 'is_summer', 'is_all_season'],
      dtype='str')
```

In [19]:
```python
# List all column names in the expenses dataset
print(f"expenses:\n{expenses.columns.tolist()}")
```

```
expenses:Index(['date', 'category', 'amount', 'notes'], dtype='str')
```

In [20]:
```python
# List all column names in the inventory dataset
print(f"inventory:\n{inventory.columns.tolist()}")
```

```
inventory:Index(['date', 'product_id', 'product_name', 'category', 'stock_level',
       'reorder_point', 'supplier', 'unit_cost', 'stock_value', 'stock_change',
       'is_suspect', 'daily_sales'],
      dtype='str')
```

In [21]:
```python
# .info() shows column names, data types, and how many non-null values each column has.
# This is useful to confirm that date columns are datetime (not plain strings)
# and that there are no unexpected nulls before we start engineering new features.
sales.info()
print()
customers.info()
print()
products.info()
print()
expenses.info()
print()
inventory.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 9989 entries, 0 to 9988
Data columns (total 19 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   sale_id          9989 non-null   str
 1   date             9989 non-null   datetime64[us]
 2   store_channel    9989 non-null   str
 3   payment_method   9989 non-null   str
 4   product_id       9989 non-null   str
 5   product_name     9989 non-null   str
 6   category         9989 non-null   str
 7   sub_category     9989 non-null   str
 8   quantity_sold    9989 non-null   int64
 9   unit_price       9989 non-null   float64
 10  discount_pct     9989 non-null   int64
 11  discount_amount  9989 non-null   float64
 12  total_sale       9989 non-null   float64
 13  cost_total       9989 non-null   float64
 14  gross_profit     9989 non-null   float64
 15  customer_id      9989 non-null   str
 16  is_promotion     9989 non-null   bool
 17  is_returned      9989 non-null   bool
 18  net_profit       9989 non-null   float64
dtypes: bool(2), datetime64[us](1), float64(6), int64(2), str(8)
memory usage: 1.3 MB
<class 'pandas.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 17 columns):
 #   Column                 Non-Null Count  Dtype
---  ------                 --------------  -----
 0   customer_id            200 non-null    str
 1   name                   200 non-null    str
 2   email                  200 non-null    str
 3   age                    198 non-null    float64
 4   gender                 200 non-null    str
 5   city                   200 non-null    str
 6   state                  200 non-null    str
 7   segment                200 non-null    str
 8   total_items_purchased  200 non-null    int64
 9   total_spent            200 non-null    float64
 10  num_orders             200 non-null    int64
```

```
 11  last_purchase_date     200 non-null    datetime64[us]
 12  favourite_category     200 non-null    str
 13  first_purchase_date    200 non-null    datetime64[us]
 14  avg_order_value        200 non-null    float64
 15  age_group              200 non-null    str
 16  derived_segment        200 non-null    str
dtypes: datetime64[us](2), float64(3), int64(2), str(10)
memory usage: 26.7 KB
<class 'pandas.DataFrame'>
RangeIndex: 25 entries, 0 to 24
Data columns (total 12 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   product_id         25 non-null     str
 1   product_name       25 non-null     str
 2   category           25 non-null     str
 3   sub_category       25 non-null     str
 4   base_price         25 non-null     float64
 5   cost_price         25 non-null     float64
 6   season             25 non-null     str
 7   profit_per_unit    25 non-null     float64
 8   profit_margin_pct  25 non-null     float64
 9   is_winter          25 non-null     bool
 10  is_summer          25 non-null     bool
 11  is_all_season      25 non-null     bool
dtypes: bool(3), float64(4), str(5)
memory usage: 2.0 KB
<class 'pandas.DataFrame'>
RangeIndex: 1322 entries, 0 to 1321
Data columns (total 4 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   date      1322 non-null   datetime64[us]
 1   category  1322 non-null   str
 2   amount    1322 non-null   float64
 3   notes     1322 non-null   str
dtypes: datetime64[us](1), float64(1), str(2)
memory usage: 41.4 KB
<class 'pandas.DataFrame'>
RangeIndex: 13675 entries, 0 to 13674
Data columns (total 12 columns):
 #   Column         Non-Null Count  Dtype
```

```
---   ------          --------------  -----
 0    date            13675 non-null  datetime64[us]
 1    product_id      13675 non-null  str
 2    product_name    13675 non-null  str
 3    category        13675 non-null  str
 4    stock_level     13675 non-null  int64
 5    reorder_point   13675 non-null  int64
 6    supplier        13675 non-null  str
 7    unit_cost       13675 non-null  float64
 8    stock_value     13675 non-null  float64
 9    stock_change    13650 non-null  float64
 10   is_suspect      13675 non-null  bool
 11   daily_sales     13675 non-null  int64
dtypes: bool(1), datetime64[us](1), float64(3), int64(3), str(4)
memory usage: 1.2 MB
```

Time-Based Features (Sales)

The `date` column in the sales table is a single timestamp. On its own, it only lets us filter rows by an exact date. By breaking it into smaller components we unlock grouping, pivoting, and trend analysis..for example, comparing weekday vs weekend sales, or spotting which quarter drives the most revenue.

New columns added: `year`, `quarter`, `month_num`, `month_name`, `day_of_week`, `is_weekend`, `days_since_start`

In [22]:
```
# Extract individual time components from the transaction date.
# Each new column gives us a different 'lens' to group or filter the data.
sales['year']        = sales['date'].dt.year                  # calendar year (e.g. 2024)
sales['quarter']     = sales['date'].dt.quarter               # 1–4
sales['month_num']   = sales['date'].dt.month                 # 1–12
sales['month_name']  = sales['date'].dt.strftime('%b')        # e.g. 'Jan', 'Feb'
sales['day_of_week'] = sales['date'].dt.day_name()            # e.g. 'Monday'
sales['is_weekend']  = sales['date'].dt.dayofweek >= 5        # True for Saturday (5) and Sunday (6)
```

In [23]:
```
# days_since_start: number of days elapsed since the very first transaction in the dataset.
# Useful for plotting trends over a continuous time axis rather than calendar dates,
# which makes it easier to fit trend lines and spot growth patterns.
sales['days_since_start'] = (sales['date'] - sales['date'].min()).dt.days
```

In [24]:
```
print(sales[['date','year','quarter','month_num','day_of_week','is_weekend','days_since_start']].head())
```

```
        date  year  quarter  month_num day_of_week  is_weekend  \
0 2024-01-01  2024        1          1      Monday       False
1 2024-01-01  2024        1          1      Monday       False
2 2024-01-01  2024        1          1      Monday       False
3 2024-01-01  2024        1          1      Monday       False
4 2024-01-01  2024        1          1      Monday       False

   days_since_start
0                 0
1                 0
2                 0
3                 0
4                 0
```

Why this matters: The raw `date` column only supports row-level filtering (e.g. 'show me sales on 2024-03-15'). Breaking it into `year`, `quarter`, `month_num`, `day_of_week`, and `is_weekend` lets us:

- Group transactions by time period without writing date logic every time
- Compare weekday vs weekend performance
- Identify seasonal patterns across quarters and months
- Use `days_since_start` as a continuous numeric variable for trend analysis

Transaction-Level Profit Metrics (Sales)

The sales table already has `gross_profit` and `net_profit`. What it lacks is the *rate*, how much of each sale ends up as profit (margin %), and how much revenue each unit of product actually generates (revenue per unit). These ratios make transactions comparable regardless of order size or product price.

New columns added: `profit_margin_pct`, `revenue_per_unit`, `break_even_sale`, `profitability`

In [25]:
```
# Transaction-level profit margin %
# Formula: net_profit / total_sale × 100
# net_profit already accounts for returns and discounts (returned rows have net_profit = 0),
# so those transactions will correctly show 0% margin.
```

```
# This is different from the product catalogue margin which is based on the base price only.
sales['profit_margin_pct'] = (sales['net_profit'] / sales['total_sale'] * 100).round(2)
```

In [26]:
```
# Revenue per unit: total revenue received divided by units sold.
# This normalises revenue so we can compare transactions with different quantities.
# For example, a 3-unit sale at a discount vs a 1-unit sale at full price.
sales['revenue_per_unit'] = (sales['total_sale'] / sales['quantity_sold']).round(2)
```

In [27]:
```
print(sales[['total_sale','quantity_sold','net_profit','profit_margin_pct','revenue_per_unit']].head())
```

```
   total_sale  quantity_sold  net_profit  profit_margin_pct  revenue_per_unit
0       80.99              1       45.99              56.78             80.99
1       50.99              1       28.99              56.85             50.99
2       40.49              1       25.49              62.95             40.49
3       19.99              1       11.99              59.98             19.99
4       80.99              1       45.99              56.78             80.99
```

`profit_margin_pct` at the transaction level is different from the product catalogue margin, it reflects the *actual* margin achieved on each specific sale after discounts and returns are applied.

`revenue_per_unit` normalises revenue across transactions with different quantities, making it possible to compare the effective selling price across orders of different sizes.

In [28]:
```
# Verify distributions look reasonable
print("profit_margin_pct stats:")
print(sales['profit_margin_pct'].describe().round(2))
print()
print("revenue_per_unit stats:")
print(sales['revenue_per_unit'].describe().round(2))
print()
print("Transactions by quarter:")
print(sales.groupby(['year','quarter']).size().reset_index(name='transactions').to_string(index=False))
```

```
profit_margin_pct stats:
count    9989.00
mean       61.35
std        14.57
min         0.00
25%        61.10
50%        63.90
75%        66.66
max        74.97
Name: profit_margin_pct, dtype: float64

revenue_per_unit stats:
count    9989.00
mean       36.04
std        19.34
min         7.99
25%        19.99
50%        33.99
75%        47.49
max        89.99
Name: revenue_per_unit, dtype: float64

Transactions by quarter:
 year  quarter  transactions
 2024        1          1831
 2024        2          1506
 2024        3          1676
 2024        4          1666
 2025        1          1764
 2025        2          1546
```

Investigating Zero-Margin Transactions

484 transactions show `profit_margin_pct = 0`. This happens when `net_profit = 0`, which is set for all returned items. Let's confirm what's driving these rows.

In [29]:
```
# Show all transactions where profit margin is 0.
# These are expected to be returns (is_returned = True) or genuinely break-even sales.
sales[sales['profit_margin_pct'] == 0]
```

Out[29]:

	sale_id	date	store_channel	payment_method	product_id	product_name	category	sub_category	quantity_sold	unit_p
27	S00028	2024-01-02	Online	Credit Card	P022	Leather Belt	All	Accessories	3	1
46	S00047	2024-01-02	Online	Mobile Pay	P020	Unisex Cap	All	Accessories	3	1
92	S00093	2024-01-03	Online	PayPal	P021	Unisex Socks (3-pack)	All	Accessories	3	
94	S00095	2024-01-03	Online	Credit Card	P021	Unisex Socks (3-pack)	All	Accessories	1	
128	S00129	2024-01-04	Physical	Credit Card	P020	Unisex Cap	All	Accessories	2	1
...	...	...	...	...	...	...	...	...	...	
9882	S09883	2025-06-25	Physical	Cash	P016	Kids T-shirt	Kids	Shirt	1	1
9915	S09916	2025-06-27	Physical	Mobile Pay	P022	Leather Belt	All	Accessories	1	1
9921	S09922	2025-06-27	Online	Credit Card	P024	Men's Swim Shorts	Men	Swimwear	3	2
9976	S09977	2025-06-30	Online	Credit Card	P020	Unisex Cap	All	Accessories	3	1
9977	S09978	2025-06-30	Online	Credit Card	P012	Women's Skinny Jeans	Women	Pants	2	5

484 rows × 28 columns

In [30]:

```
# Which products appear most often in zero-margin transactions?
# High counts here point to high-return products worth investigating.
sales[sales['profit_margin_pct'] == 0]['product_name'].value_counts()
```

Out[30]:
```
product_name
Unisex Cap                42
Leather Belt              38
Canvas Tote Bag           37
Unisex Socks (3-pack)     35
Men's Premium T-shirt     25
Women's Skinny Jeans      25
Men's Basic T-shirt       21
Women's Swimsuit          21
Men's Slim Jeans          21
Women's Skirt             20
Women's Puffer Jacket     19
Women's Floral Dress      18
Men's Swim Shorts         18
Women's Blouse            18
Women's Cardigan          16
Men's Puffer Jacket       16
Men's Hoodie              15
Men's Polo Shirt          14
Women's Hoodie            13
Women's Casual Dress      13
Kids T-shirt              12
Men's Chino Pants         11
Kids Jeans                 7
Kids Dress                 7
Kids Jacket                2
Name: count, dtype: int64
```

In [31]:
```
# Are zero-margin transactions linked to promotions?
# If most are False, returns are not promo-driven — they may be quality or sizing issues.
sales[sales['profit_margin_pct'] == 0]['is_promotion'].value_counts()
```

Out[31]:
```
is_promotion
False    357
True     127
Name: count, dtype: int64
```

In [32]:
```
# Create a boolean flag: True if the transaction generated zero net profit.
# This makes it easy to filter, count, or group break-even rows without repeating the condition.
sales['break_even_sale'] = sales['net_profit'] == 0
```

In [33]:
```
# What share of all transactions are break-even?
# .mean() on a boolean column returns the proportion of True values (0.0 to 1.0).
break_even_pct = sales['break_even_sale'].mean() * 100
print(f"Break-even transactions: {break_even_pct:.2f}% of all sales")
```

Out[33]:
```
np.float64(0.04845329862849134)
```

In [34]:
```
# Look at the discount distribution for zero-margin rows.
# If mean discount is low, returns are the main cause — not heavy discounting.
sales[sales['profit_margin_pct'] == 0]['discount_pct'].describe()
```

Out[34]:
```
count    484.000000
mean       5.165289
std        5.904130
min        0.000000
25%        0.000000
50%        5.000000
75%       10.000000
max       20.000000
Name: discount_pct, dtype: float64
```

Profitability Tiers

Bucketing transactions into margin bands (e.g. 'Very High', 'High') makes it easier to count and visualise how many sales fall into each category, rather than looking at raw percentages alone.

In [44]:
```
# Bucket each transaction into a profitability tier based on its profit margin %.
# pd.cut assigns each value to a range defined by 'bins'.
# Labels correspond to those ranges left-to-right:
#   -1 to  0 → Loss/Break-even  (returned or zero-profit sales)
#    0 to 20 → Low              (heavily discounted sales)
#   20 to 40 → Medium
#   40 to 60 → High
#   60 to 100→ Very High        (full-price, no return)
sales['profitability'] = pd.cut(
    sales['profit_margin_pct'],
    bins=[-1, 0, 20, 40, 60, 100],
    labels=['Loss/Break-even', 'Low', 'Medium', 'High', 'Very High']
)
```

In [45]:
```
sales["profitability"].value_counts()
```

Out[45]:
```
profitability
Very High          7900
High               1605
Loss/Break-even     484
Low                   0
Medium                0
Name: count, dtype: int64
```

Product Feature Join (Sales ← Products)

Each sales row contains a `product_id` but no product-level attributes like season or catalogue margin. We join a small slice of the products table onto the sales table so every transaction carries these context columns.

We also compute **margin erosion** ~ the gap between the catalogue (theoretical) margin and the margin actually realised on each transaction. This tells us exactly how much discounts and returns cost in margin points.

New columns added: `season`, `catalogue_margin_pct`, `is_winter`, `is_summer`, `cost_price`, `margin_erosion`

In [46]:
```
# Select only the columns we need from the products table — avoids cluttering the sales table.
# We rename profit_margin_pct to catalogue_margin_pct to distinguish it from the
# transaction-level profit_margin_pct we calculated above.
product_features = products[['product_id', 'season', 'profit_margin_pct', 'is_winter', 'is_summer', 'cost_price']]
product_features = product_features.rename(columns={'profit_margin_pct': 'catalogue_margin_pct'})

# Left join: every sales row is kept; product attributes are added where product_id matches.
# how='left' ensures we don't lose any sales rows if a product_id is somehow missing from products.
sales = sales.merge(product_features, on='product_id', how='left')
```

In [50]:
```
# Verify the join — all rows should have a season tag
missing_season = sales['season'].isna().sum()
print(f"Transactions with no season tag after join: {missing_season}")
print()
print("Season distribution across transactions:")
print(sales['season'].value_counts().to_string())
```

```
Transactions with no season tag after join: 0

Season distribution across transactions:
season
All       6233
Summer    1888
Winter    1868
```

In [51]:
```python
# Check: catalogue_margin_pct vs realised profit_margin_pct
# The gap tells us how much discounts and returns erode the catalogue margin in practice
sales['margin_erosion'] = (sales['catalogue_margin_pct'] - sales['profit_margin_pct']).round(2)

print("Margin erosion (catalogue margin minus realised margin):")
print(sales['margin_erosion'].describe().round(2))
print()
```

```
Margin erosion (catalogue margin minus realised margin):
count    9989.00
mean        5.16
std        14.08
min        -0.05
25%         0.01
50%         1.80
75%         4.01
max        75.00
Name: margin_erosion, dtype: float64
```

In [52]:
```python
print("Avg margin erosion by discount tier:")
print(sales.groupby('discount_pct')['margin_erosion'].mean().round(2).to_string())
```

```
Avg margin erosion by discount tier:
discount_pct
0      3.37
5      4.73
10     6.61
15     8.96
20    10.87
```

Each transaction now carries the product's **catalogue margin** (what we expect to earn at full price) alongside the **actual realised margin** (what we actually earned after discounts and returns).

`margin_erosion` = catalogue_margin_pct − profit_margin_pct

A higher number means more margin was lost on that transaction. The breakdown by discount tier shows a clear pattern: every 5 percentage-point increase in discount adds roughly 2 extra margin points of erosion.

Customer RFM Scoring

RFM (Recency, Frequency, Monetary) is the standard retail framework for scoring customer value based on actual purchase behaviour — not demographics.

Dimension	What it measures	Better =
Recency (R)	Days since last purchase	Lower
Frequency (F)	Number of orders placed	Higher
Monetary (M)	Total amount spent	Higher

Each dimension is scored 1–4 using quartiles (4 = best quartile). The three scores are summed to give a combined **RFM score of 3–12**, which maps to five business segments:

Score range	Segment
11–12	Champion
9–10	Loyal
7–8	Potential
5–6	At Risk
3–4	Lost

Why RFM matters: RFM is behaviour-based and forward-looking. It can identify customers about to churn, a customer who used to buy frequently and spend a lot (high F and M) but hasn't purchased recently (low R) is a win-back opportunity *before* they leave entirely.

In [53]:
```
# Reference date: one day after the last transaction in the dataset
# Using the dataset end date rather than today ensures reproducibility
reference_date = pd.Timestamp('2025-07-01')

# Recency: days since last purchase (lower = more recent = better)
customers['recency_days'] = (reference_date - customers['last_purchase_date']).dt.days

# Tenure: days from first to last purchase — measures loyalty window
customers['tenure_days'] = (customers['last_purchase_date'] - customers['first_purchase_date']).dt.days
```

In [54]:
```
print(customers[['customer_id','first_purchase_date','last_purchase_date','recency_days','tenure_days']].head())
```

```
  customer_id first_purchase_date last_purchase_date  recency_days  \
0        C001          2024-01-29         2025-06-18            13
1        C002          2024-01-03         2025-06-25             6
2        C003          2024-01-12         2025-06-20            11
3        C004          2024-01-06         2025-06-26             5
4        C005          2024-01-09         2025-06-23             8

   tenure_days
0          506
1          539
2          525
3          537
4          531
```

In [55]:
```
# Score each customer on R, F, and M using quartiles (pd.qcut splits into 4 equal-sized groups).
# We use .rank(method='first') before qcut to handle tied values cleanly — this assigns
# each tied value a unique rank so qcut can split them evenly.

# R score: most recent buyer should get the highest score (4), so we reverse the labels.
# Lower recency_days = more recent = better, so label order goes [4, 3, 2, 1].
customers['R_score'] = pd.qcut(customers['recency_days'].rank(method='first', ascending=True),
                               q=4, labels=[4, 3, 2, 1]).astype(int)

# F score: more orders = better = higher score, so labels go [1, 2, 3, 4] (ascending).
customers['F_score'] = pd.qcut(customers['num_orders'].rank(method='first'),
                               q=4, labels=[1, 2, 3, 4]).astype(int)

# M score: higher spend = better = higher score.
customers['M_score'] = pd.qcut(customers['total_spent'].rank(method='first'),
```

```
                                q=4, labels=[1, 2, 3, 4]).astype(int)

# Combined RFM score: sum of the three scores, range 3 (worst) to 12 (best).
customers['RFM_score'] = customers['R_score'] + customers['F_score'] + customers['M_score']
```

In [56]:
```
print("RFM scores calculated.")
print(customers[['customer_id','recency_days','num_orders','total_spent',
                 'R_score','F_score','M_score','RFM_score']].head(10).to_string(index=False))
```

```
RFM scores calculated.
customer_id  recency_days  num_orders  total_spent  R_score  F_score  M_score  RFM_score
       C001            13          57      3108.42        2        4        3          9
       C002             6          67      6903.24        3        4        4         11
       C003            11          43      2501.08        2        1        1          4
       C004             5          48      2777.74        3        2        2          7
       C005             8          60      3634.30        2        4        3          9
       C006             8          53      2708.72        2        3        1          6
       C007             2          40      2642.09        4        1        1          6
       C008             3          61      5549.68        4        4        4         12
       C009             4          54      2941.64        3        3        2          8
       C010            17          61      3681.88        2        4        3          9
```

In [57]:
```
# Map the combined RFM score to a human-readable business segment.
# The score range is 3–12 (all values covered), but we add .fillna('Unknown') as a
# safety net in case any edge case produces a score outside this range.
rfm_segment_map = {
    12: 'Champion',
    11: 'Champion',
    10: 'Loyal',
    9:  'Loyal',
    8:  'Potential',
    7:  'Potential',
    6:  'At Risk',
    5:  'At Risk',
    4:  'Lost',
    3:  'Lost'
}
customers['RFM_segment'] = customers['RFM_score'].map(rfm_segment_map).fillna('Unknown')
```

In [58]:
```
print(customers['RFM_segment'].value_counts().to_string())
print()
print("Avg metrics by RFM segment:")
```

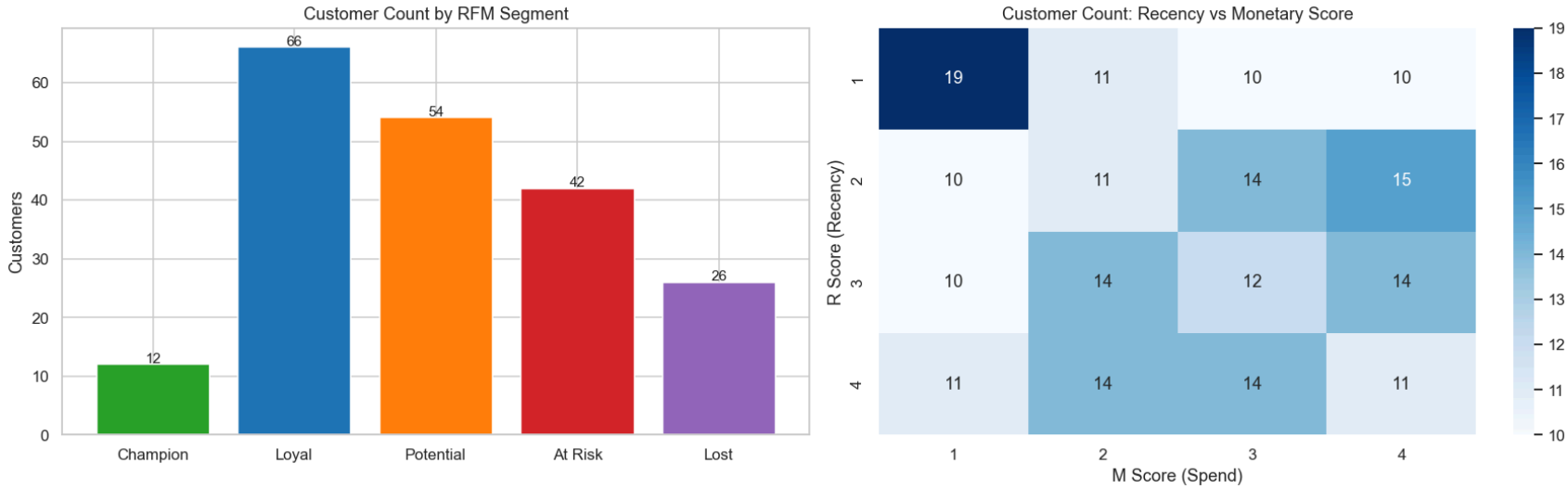
```python
print(customers.groupby('RFM_segment').agg(
    count        = ('customer_id', 'count'),
    avg_recency  = ('recency_days', 'mean'),
    avg_orders   = ('num_orders', 'mean'),
    avg_spent    = ('total_spent', 'mean')
).round(1).to_string())
```

```
RFM_segment
Loyal        66
Potential    54
At Risk      42
Lost         26
Champion     12

Avg metrics by RFM segment:
             count  avg_recency  avg_orders  avg_spent
RFM_segment
At Risk         42         13.6        45.4     2693.3
Champion        12          3.2        63.9     5026.6
Lost            26         25.2        41.8     2320.9
Loyal           66          6.2        53.6     3910.6
Potential       54         13.3        49.9     3337.5
```

In [59]:

```python
# Visualise RFM segment distribution and score heatmap
fig, axes = plt.subplots(1, 2, figsize=(16, 5))

seg_order = ['Champion', 'Loyal', 'Potential', 'At Risk', 'Lost']
seg_counts = customers['RFM_segment'].value_counts().reindex(seg_order)
colors = [GREEN, BLUE, ORANGE, RED, PURPLE]

axes[0].bar(seg_counts.index, seg_counts.values, color=colors)
axes[0].set_title("Customer Count by RFM Segment")
axes[0].set_ylabel("Customers")
for bar, val in zip(axes[0].patches, seg_counts.values):
    axes[0].text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.3,
                 str(val), ha='center', fontsize=10)

# R vs M score heatmap — shows whether recency and monetary value are correlated
rfm_pivot = customers.groupby(['R_score','M_score']).size().reset_index(name='count')
rfm_heatmap = rfm_pivot.pivot(index='R_score', columns='M_score', values='count').fillna(0)
sns.heatmap(rfm_heatmap, annot=True, fmt='.0f', cmap='Blues', ax=axes[1])
axes[1].set_title("Customer Count: Recency vs Monetary Score")
```

```
axes[1].set_xlabel("M Score (Spend)")
axes[1].set_ylabel("R Score (Recency)")

plt.tight_layout()
plt.show()
```



Every customer now has an **RFM_segment** label grounded in actual purchase behaviour.

- Champions (score 10–12): bought recently, buy often, spend most → reward and retain.
- Loyal (score 8–9): consistent buyers, strong spend → upsell and cross-sell.
- Potential (score 6–7): moderate engagement → nurture with targeted campaigns.
- At Risk (score 4–5): used to buy but recency has dropped → win-back campaigns needed.
- Lost (score 3): haven't bought recently, low frequency → low ROI to reactivate.

The **R vs M heatmap** shows whether StyleSense's highest spenders are still buying recently,if they cluster in low-R cells, there is a churn problem among the most valuable customers.

Inventory: Stock Health & Days Until Stockout

Raw inventory data tells us the current stock level but not whether that level is *safe* or *dangerously low*. We add two perspectives:

1. `stock_health` - how far above (or below) the reorder threshold the stock is. A ratio of 1.0 means stock is exactly at the reorder point. Below 1.0 = critical.

2. `days_until_stockout` - at the current sales pace, how many days until the product runs out of stock. Uses a 7-day rolling average to smooth out zero-sale days.

New columns added: `stock_health`, `is_near_stockout`, `rolling_avg_sales`, `days_until_stockout`

In [60]:
```
# Stock health ratio: stock_level divided by reorder_point.
# Interpretation:
#   > 1.0  → safe (stock is above the reorder threshold)
#   = 1.0  → at the reorder threshold — time to order now
#   < 1.0  → critical (stock has already fallen below the safe minimum)
inventory['stock_health'] = (inventory['stock_level'] / inventory['reorder_point']).round(2)

# Boolean flag: True if the product needs attention (at or below reorder point)
inventory['is_near_stockout'] = inventory['stock_level'] <= inventory['reorder_point']
```

In [61]:
```
print("Stock health summary:")
print(inventory['stock_health'].describe().round(2))
print()
print(f"Near-stockout records: {inventory['is_near_stockout'].sum():,} "
      f"({inventory['is_near_stockout'].mean()*100:.1f}% of all records)")
```

```
Stock health summary:
count    13675.00
mean         2.63
std          1.00
min          1.00
25%          1.78
50%          2.55
75%          3.35
max          5.00
Name: stock_health, dtype: float64

Near-stockout records: 96 (0.7% of all records)
```

In [62]:
```
# Days until stockout: how long current stock will last at the current sales pace.
# We use a 7-day rolling average of daily_sales to smooth out zero-sale days
# (days with no sales for a product shouldn't make it look like it will never sell out).
```

```python
inventory = inventory.sort_values(['product_id', 'date']).reset_index(drop=True)

# rolling(7, min_periods=1) computes a 7-day trailing average; min_periods=1 ensures
# the first few rows (with fewer than 7 days of history) still get a value.
inventory['rolling_avg_sales'] = (
    inventory.groupby('product_id')['daily_sales']
    .transform(lambda x: x.rolling(7, min_periods=1).mean())
)

# If rolling avg is 0 (product had no sales in the window), we set days_until_stockout
# to 999 — meaning no near-term risk can be estimated from recent sales history.
inventory['days_until_stockout'] = inventory.apply(
    lambda row: round(row['stock_level'] / row['rolling_avg_sales'], 1)
    if row['rolling_avg_sales'] > 0 else 999,
    axis=1
)
```

In [63]:
```python
print(inventory[['product_name','date','stock_level','rolling_avg_sales','days_until_stockout']].head(10).to_string(
```

```
      product_name       date  stock_level  rolling_avg_sales  days_until_stockout
Men's Basic T-shirt 2024-01-01          200           0.000000                999.0
Men's Basic T-shirt 2024-01-02          199           0.500000                398.0
Men's Basic T-shirt 2024-01-03          195           1.666667                117.0
Men's Basic T-shirt 2024-01-04          195           1.250000                156.0
Men's Basic T-shirt 2024-01-05          195           1.000000                195.0
Men's Basic T-shirt 2024-01-06          195           0.833333                234.0
Men's Basic T-shirt 2024-01-07          192           1.142857                168.0
Men's Basic T-shirt 2024-01-08          187           1.857143                100.7
Men's Basic T-shirt 2024-01-09          187           1.714286                109.1
Men's Basic T-shirt 2024-01-10          184           1.571429                117.1
```

In [65]:
```python
#Summarise by product: avg days until stockout and avg stock health
inventory_summary = inventory.groupby(['product_id','product_name','category']).agg(
    avg_stock_health     = ('stock_health', 'mean'),
    min_stock_health     = ('stock_health', 'min'),
    near_stockout_days   = ('is_near_stockout', 'sum'),
    avg_days_to_stockout = ('days_until_stockout', 'mean')
).reset_index()

inventory_summary['avg_stock_health']     = inventory_summary['avg_stock_health'].round(2)
inventory_summary['avg_days_to_stockout'] = inventory_summary['avg_days_to_stockout'].round(1)
inventory_summary = inventory_summary.sort_values('avg_stock_health')
```

```python
print("Inventory Health by Product (sorted by most at-risk):")
print(inventory_summary.to_string(index=False))
```

Inventory Health by Product (sorted by most at-risk):

product_id	product_name	category	avg_stock_health	min_stock_health	near_stockout_days	avg_days_to_stockout
P004	Men's Chino Pants	Men	2.39	1.00	4	101.7
P006	Men's Puffer Jacket	Men	2.41	1.00	1	201.3
P020	Unisex Cap	All	2.42	1.00	7	57.9
P012	Women's Skinny Jeans	Women	2.44	1.02	0	116.3
P021	Unisex Socks (3-pack)	All	2.44	1.00	2	56.6
P015	Women's Cardigan	Women	2.50	1.00	11	221.1
P014	Women's Puffer Jacket	Women	2.52	1.00	1	250.0
P001	Men's Basic T-shirt	Men	2.53	1.00	4	108.7
P010	Women's Blouse	Women	2.55	1.00	4	120.2
P022	Leather Belt	All	2.56	1.00	6	64.9
P007	Men's Polo Shirt	Men	2.58	1.00	5	213.7
P003	Men's Slim Jeans	Men	2.59	1.00	6	118.1
P005	Men's Hoodie	Men	2.59	1.00	3	221.8
P023	Canvas Tote Bag	All	2.63	1.00	7	54.5
P002	Men's Premium T-shirt	Men	2.63	1.00	7	152.0
P011	Women's Skirt	Women	2.64	1.00	1	253.0
P019	Kids Dress	Kids	2.64	1.05	0	533.3
P013	Women's Hoodie	Women	2.69	1.00	3	253.1
P016	Kids T-shirt	Kids	2.71	1.05	0	449.1
P025	Women's Swimsuit	Women	2.73	1.00	1	19

```
4.6
     P017            Kids Jeans     Kids              2.81              1.02                   0                 46
6.0
     P008  Women's Floral Dress    Women              2.81              1.00                   2                 26
0.4
     P009  Women's Casual Dress    Women              2.83              1.00                   2                 14
7.4
     P024     Men's Swim Shorts      Men              2.95              1.00                  19                 28
4.6
     P018           Kids Jacket     Kids              3.02              1.02                   0                 57
3.7
```

- **stock_health**: ratio of current stock to reorder point, a value of 1.0 means you are exactly at the threshold; below 1.0 means you are already below the reorder point.
- **is_near_stockout**: boolean flag, simpler to filter than computing the ratio each time.
- **days_until_stockout**: estimates how many days of stock remain based on the rolling average daily sales rate, a forward-looking metric that powers prescriptive recommendations.

Expenses: Fixed vs Variable Classification

P&L analysis requires separating costs into two types:

- **Fixed costs**: stay the same regardless of how much we sell (e.g. Rent, Insurance). They form a baseline that must be covered before any profit is made.

- **Variable costs**: scale with business activity (e.g. Cost of Goods Sold, Marketing, Shipping). They should rise in high-revenue months and fall in quiet ones.

Treating these two types the same makes cost analysis misleading. A spike in variable costs during a high-sales month is expected. A spike in fixed costs is a red flag.

New columns added: `is_fixed_cost`, `cost_type`, `year`, `month_num`, `quarter`

In [66]:
```
# Define which expense categories are fixed vs variable.
# Fixed costs: the same amount is owed every month regardless of sales volume.
# Variable costs: amounts fluctuate based on business activity.
```

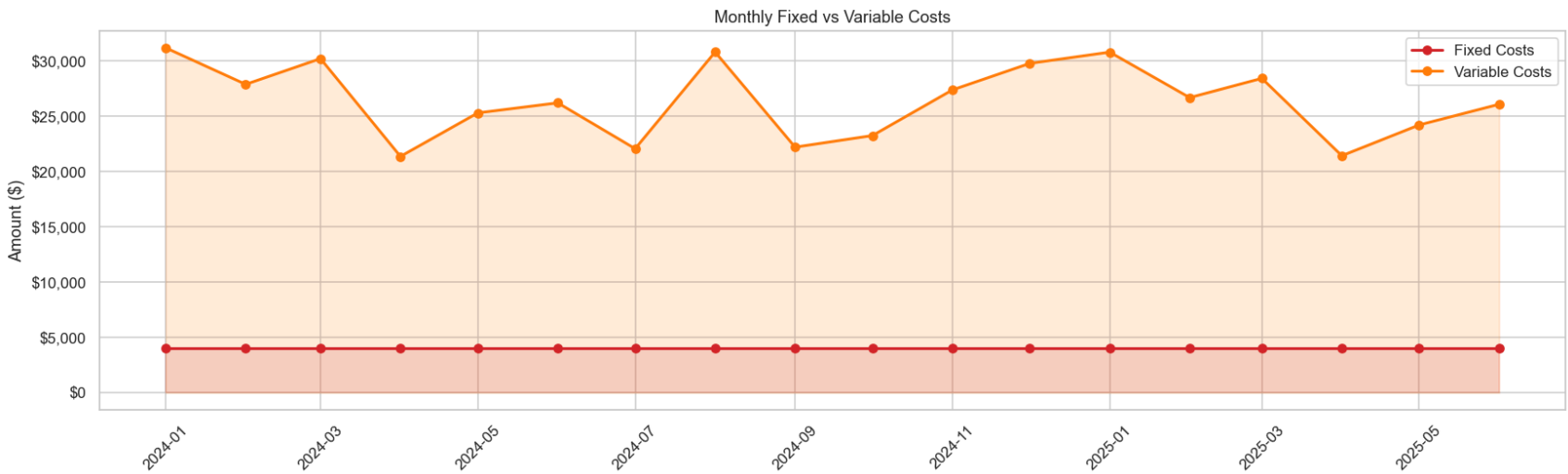
```python
fixed_categories = ['Rent', 'Insurance', 'Software & Subscriptions']

# Create a boolean column: True if the category is a fixed cost
expenses['is_fixed_cost'] = expenses['category'].isin(fixed_categories)

# Create a readable label column for use in charts and groupby operations
expenses['cost_type'] = expenses['is_fixed_cost'].map({True: 'Fixed', False: 'Variable'})

# Extract time components from the expense date for monthly and quarterly trend analysis
expenses['year']      = expenses['date'].dt.year
expenses['month_num'] = expenses['date'].dt.month
expenses['quarter']   = expenses['date'].dt.quarter
```

In [67]:
```python
print(expenses.groupby('cost_type')['amount'].agg(['sum','mean','count']).round(2).to_string())
print()
print("Category classification:")
print(expenses.groupby(['category','cost_type'])['amount'].sum().round(2).reset_index().to_string(index=False))
```

```
                 sum     mean  count
cost_type
Fixed       72900.00  1350.00     54
Variable   474579.42   374.27   1268

Category classification:
                category cost_type    amount
      Cost of Goods Sold  Variable 249989.00
               Insurance     Fixed   7200.00
               Marketing  Variable  15064.49
                 Payroll  Variable 187300.32
                    Rent     Fixed  63000.00
    Shipping & Packaging  Variable  15870.57
Software & Subscriptions     Fixed   2700.00
               Utilities  Variable   6355.04
```

In [68]:
```python
# Monthly fixed vs variable cost trend
monthly_cost_type = expenses.groupby(['date', 'cost_type'])['amount'].sum().reset_index()

# Aggregate to month level
monthly_cost_type['month'] = monthly_cost_type['date'].dt.to_period('M')
monthly_pivot = monthly_cost_type.groupby(['month','cost_type'])['amount'].sum().reset_index()
monthly_pivot['month_dt'] = monthly_pivot['month'].dt.to_timestamp()
```

```
fixed_data    = monthly_pivot[monthly_pivot['cost_type'] == 'Fixed']
variable_data = monthly_pivot[monthly_pivot['cost_type'] == 'Variable']

fig, ax = plt.subplots(figsize=(16, 5))
ax.plot(fixed_data['month_dt'],    fixed_data['amount'],    label='Fixed Costs',    color=RED,  marker='o', linewidth
ax.plot(variable_data['month_dt'], variable_data['amount'], label='Variable Costs', color=ORANGE, marker='o', linewi
ax.fill_between(fixed_data['month_dt'],    fixed_data['amount'],    alpha=0.15, color=RED)
ax.fill_between(variable_data['month_dt'], variable_data['amount'], alpha=0.15, color=ORANGE)
ax.set_title("Monthly Fixed vs Variable Costs")
ax.set_ylabel("Amount ($)")
ax.yaxis.set_major_formatter(plt.FuncFormatter(lambda x, _: f"${x:,.0f}"))
ax.legend()
ax.tick_params(axis='x', rotation=45)
plt.tight_layout()
plt.show()
```



How to read the chart:

- **Fixed costs** (red line) should form a flat baseline each month. A spike indicates a one-off or incorrectly classified cost worth investigating.
- **Variable costs** (orange line) should broadly track sales volume — higher in busy months, lower in quiet ones. If variable costs are rising *faster* than revenue, it means margins are being compressed and the business is spending more to earn the same.

Save Enriched Datasets

All five datasets are saved as new CSV files with the `_featured` suffix. These are the files that will be used in the analysis and visualisation notebooks. The original `_clean` files are left unchanged.

In [69]:
```
# Export all enriched datasets to CSV.
# index=False prevents pandas from writing the row numbers as a column in the file.
sales.to_csv('sales_featured.csv', index=False)
customers.to_csv('customers_featured.csv', index=False)
expenses.to_csv('expenses_featured.csv', index=False)
inventory.to_csv('inventory_featured.csv', index=False)

# Products had no new columns added in this notebook beyond what was done in cleaning.
# We still save it under the consistent _featured naming so all downstream notebooks
# only need to import from _featured files.
products.to_csv('products_featured.csv', index=False)
```

In [70]:
```
# Confirm the final shape and column list for each enriched dataset.
# Compare these against the original shapes printed at the top of the notebook
# to verify that all new columns were added as expected.
print(f"  sales_featured.csv     : {sales.shape}    columns: {list(sales.columns)}")
print()
print(f"  customers_featured.csv : {customers.shape}  columns: {list(customers.columns)}")
print()
print(f"  products_featured.csv  : {products.shape}   columns: {list(products.columns)}")
print()
print(f"  expenses_featured.csv  : {expenses.shape}  columns: {list(expenses.columns)}")
print()
print(f"  inventory_featured.csv : {inventory.shape} columns: {list(inventory.columns)}")
```

```
  sales_featured.csv     : (9989, 36)    columns: ['sale_id', 'date', 'store_channel', 'payment_method', 'product_i
d', 'product_name', 'category', 'sub_category', 'quantity_sold', 'unit_price', 'discount_pct', 'discount_amount', 'to
tal_sale', 'cost_total', 'gross_profit', 'customer_id', 'is_promotion', 'is_returned', 'net_profit', 'year', 'quarte
r', 'month_num', 'month_name', 'day_of_week', 'is_weekend', 'days_since_start', 'profit_margin_pct', 'revenue_per_uni
t', 'break_even_sale', 'profitability', 'season', 'catalogue_margin_pct', 'is_winter', 'is_summer', 'cost_price', 'ma
rgin_erosion']

  customers_featured.csv : (200, 24)  columns: ['customer_id', 'name', 'email', 'age', 'gender', 'city', 'state', 'se
gment', 'total_items_purchased', 'total_spent', 'num_orders', 'last_purchase_date', 'favourite_category', 'first_purc
hase_date', 'avg_order_value', 'age_group', 'derived_segment', 'recency_days', 'tenure_days', 'R_score', 'F_score',
'M_score', 'RFM_score', 'RFM_segment']

  products_featured.csv  : (25, 12)   columns: ['product_id', 'product_name', 'category', 'sub_category', 'base_pric
e', 'cost_price', 'season', 'profit_per_unit', 'profit_margin_pct', 'is_winter', 'is_summer', 'is_all_season']

  expenses_featured.csv  : (1322, 9)  columns: ['date', 'category', 'amount', 'notes', 'is_fixed_cost', 'cost_type',
'year', 'month_num', 'quarter']

  inventory_featured.csv : (13675, 16) columns: ['date', 'product_id', 'product_name', 'category', 'stock_level', 're
order_point', 'supplier', 'unit_cost', 'stock_value', 'stock_change', 'is_suspect', 'daily_sales', 'stock_health', 'i
s_near_stockout', 'rolling_avg_sales', 'days_until_stockout']
```

In [ ]: